Assignment 3 Jacobi Iteration

- Simpler than SOR (Successive Over-relaxation)
- Do not need to keep track of global delta for converge. After predefined number of iteration, algorithm terminates.

1.0	0.0	0.0	0.0	0.0	0.0
1.0	0.0	0.0	0.0	0.0	0.0
1.0	0.0	0.0	0.0	0.0	0.0
1.0	0.0	0.0	0.0	0.0	0.0
1.0	0.0	0.0	0.0	0.0	0.0
1.0	0.0	0.0	0.0	0.0	0.0

Interior value
Boundary value

In each iteration, we replace each interior cell with the average of its 8 nearest neighbors and itself (convolution).

In this assignment, we use weighted average.
*avg = a * North + b * East + c * South + d * West;*
(a + b + c + d = 1.0)

Assignment 3 Jacobi Iteration

```
//Serial algorithm
//input: OLD array contains the data to be
  processed
// and preDefinedIterations
1.  for( i = 0; i<preDefinedIterations; i ++)
2.  {
3.      Compute the average of 8 neighbors for each
        value in OLD array, store the result into NEW at
        the corresponding position.

4.      at the end of each iteration, OLD = NEW
5.  }
```

```
float * serial (float *a1, float*a2, int width, int height, int passes) {
  int i,j,p;
  float * old=a1;
  float * New=a2;
  float * temp;
  for(p=0; p<passes; p++){
    for(i=1; i<height-1; i++){
      for(j=1; j<width-1; j++){
        New[i*width +j] = (
                    0.2f * old[i*width + j] +

                    0.1f * old[(i-1) * width +  j   ] +      //N
                    0.1f * old[(i-1) * width + (j+1)] +      //NE
                    0.1f * old[ i    * width + (j+1)] +      //E
                    0.1f * old[(i+1) * width + (j+1)] +      //SE
                    0.1f * old[(i+1) * width +  j   ] +      //S
                    0.1f * old[(i+1) * width + (j-1)] +      //SW
                    0.1f * old[ i    * width + (j-1)] +      //W
                    0.1f * old[(i-1) * width + (j-1)]        //NW
                   ) * 0.95f;
      }
    }
    temp = New;
    New = old;
    old = temp;
  }
  return old;
}
```

Assignment 3 Jacobi Iteration

– Summary for parallel Jacobi iteration

1. Load required elements from old array (device global memory) into shared memory.
2. We want aligned access when possible, e.g. with the 3-row technique, row length is the blockwidth + 2.
3. Use if statement to handle “spare change” at the end of a row.

Wrap up for using shared mem

1. shared mem is fast, 100X than global mem
2. shared mem is declared only once in your kernel, but the FACT is that each block has the declared shared memory associated with it.

   It is prohibitive to declare NumBlock shared memory for each thread blocks separately.
3. Hence, your mind should witch back and forth between “per thread” and “per block”. Note that if you use shared mem, each thread is also associated with the shared mem for that block.

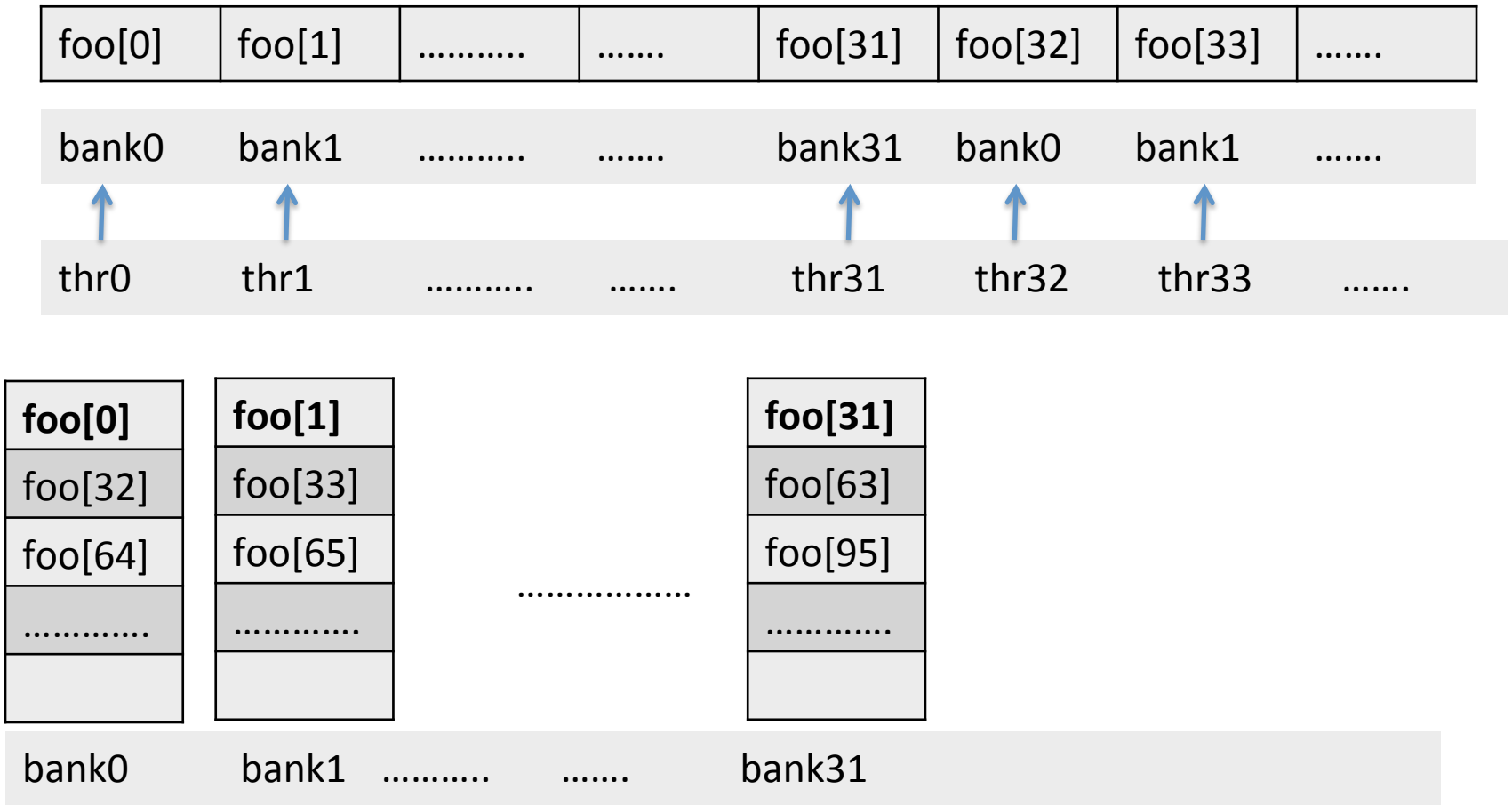
Bank conflicts

1. Occurs when accessing shared memory.
2. Shared memory is organized into banks that can be accessed simultaneously to maximize bandwidth.
3. In compute capability 2.0(fermi),
   - 32 banks that are each 32 bits wide (64 bits for Kepler capability 3.0)
   - can access 1024 bits at a time
   - same as global memory (128 bytes).
   - Kepler GPUs are configurable where either successive 32-bit words OR 64-bit words are assigned to successive banks.

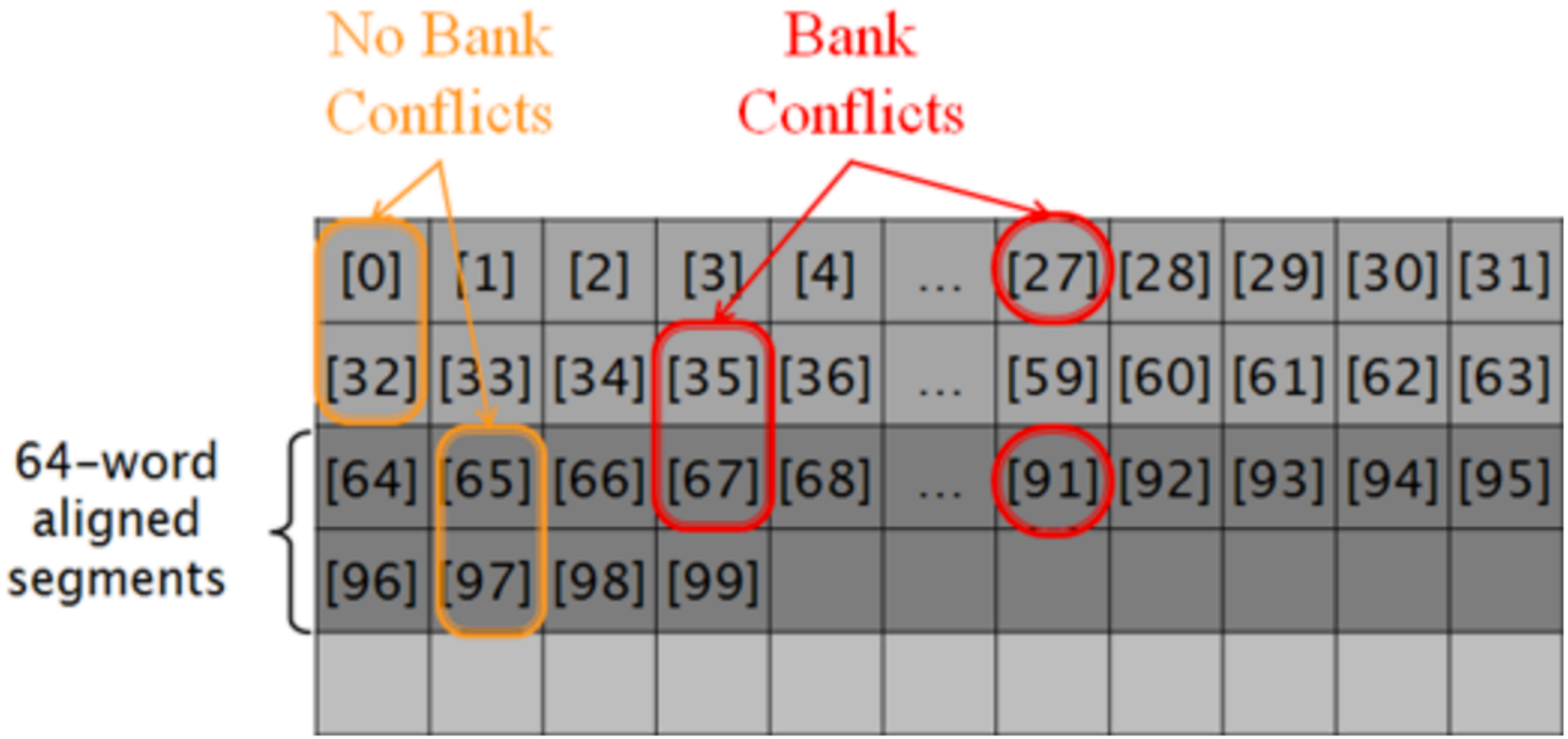
– http://acceleware.com/blog/maximizing-shared-memory-bandwidth-nvidia-kepler-gpus

Bank conflicts for bank of width 32 bits

1. __shared__ float foo[1024]



Bank conflicts for bank of width 64 bits



Bank conflicts

4. Definition: if K>1 threads in **a warp** access different 32 bits words within the same bank, we have a bank conflicts, if bank width is 32 bits.
5. If the K threads access bytes within the same 32 bit word,
   - values can be broadcast to threads when reading.
   - each byte is written to by only one thread (race condition)
6. **So if each thread in a warp accesses successive 32bit values there are no bank conflicts.**

Memory Coalescing

1. Applies only to compute capability 1.x, or cuda 2.0. More recent architectures and cuda 3.0 have more sophisticated global memory access and in fact "coalesced global loads" are not even profiled for these chips.
    - But you have to know what is going on underlying.
2. Occurs when accessing all off-chip mem.
3. E.g. For compute capability 1.3, for 4-byte values, the 16 memory access(half warp access global mem) can be coalesced, combining into 1 read of length of 64 bytes.

Memory Coalescing

4. For compute capability 2.0(fermi and newer),
   - can do 128 bytes access.
   - **a cache** on top of global device mem ensures that all accesses to global mem are coalesced.
   - An non-aligned access may still incur an extra memory access.

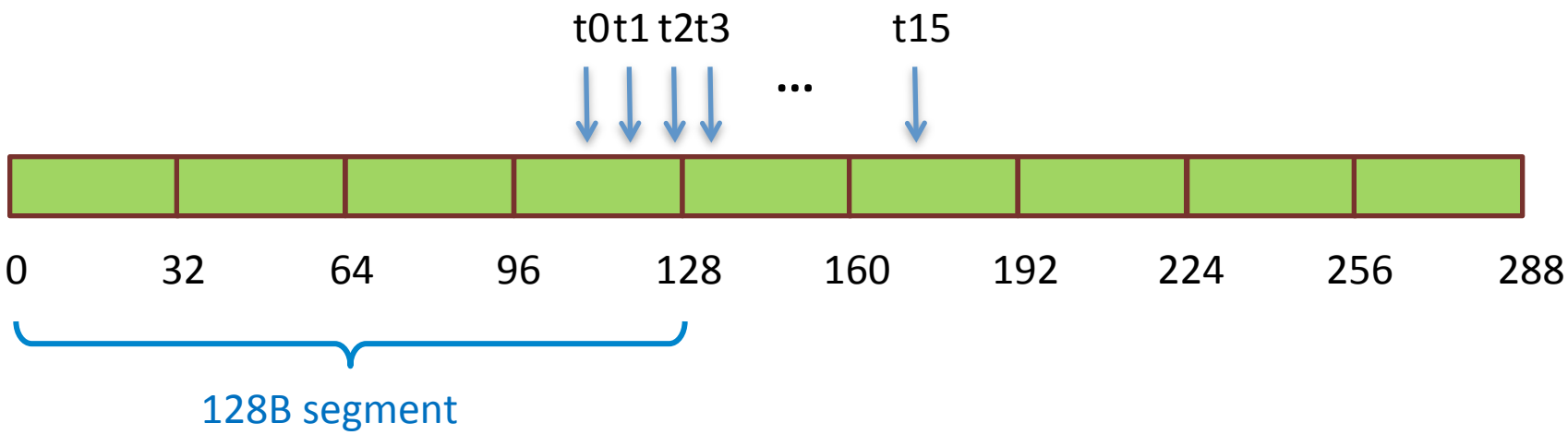
Memory Coalescing

- Off-chip memory is accessed in chunks
  - Even if you read only a single word
  - If you don't use whole chunk, bandwidth is wasted
- Chunks are aligned to multiples of 32/64/128 bytes
  - Unaligned accesses will cost more

Memory Coalescing

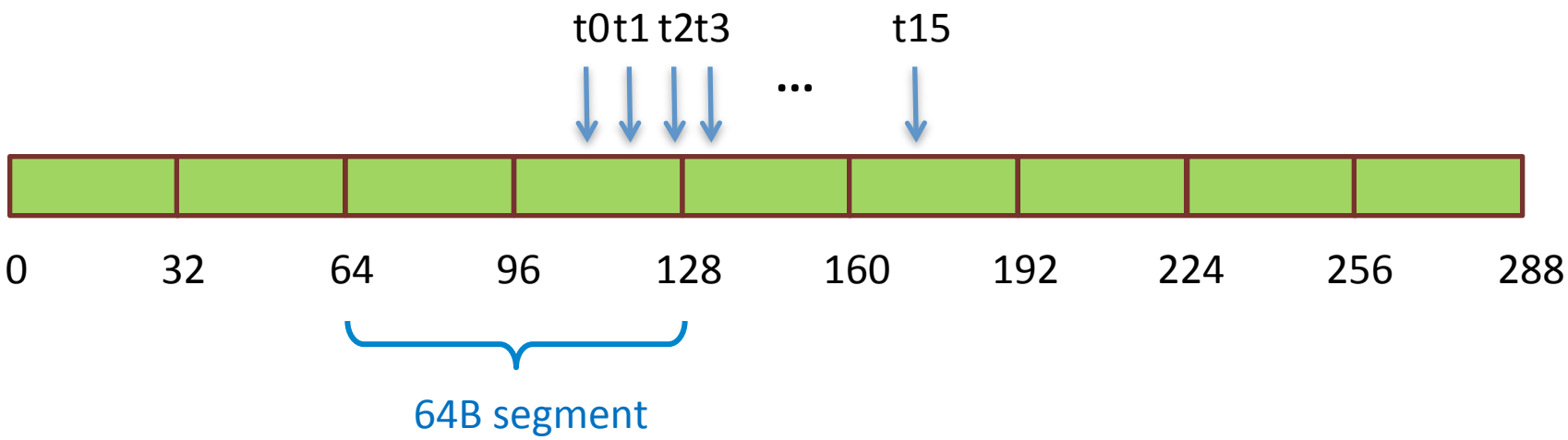
Threads 0-15 access 4-byte words at addresses 116-176

- Thread 0 is lowest active, accesses address 116
- 128-byte segment: 0-127



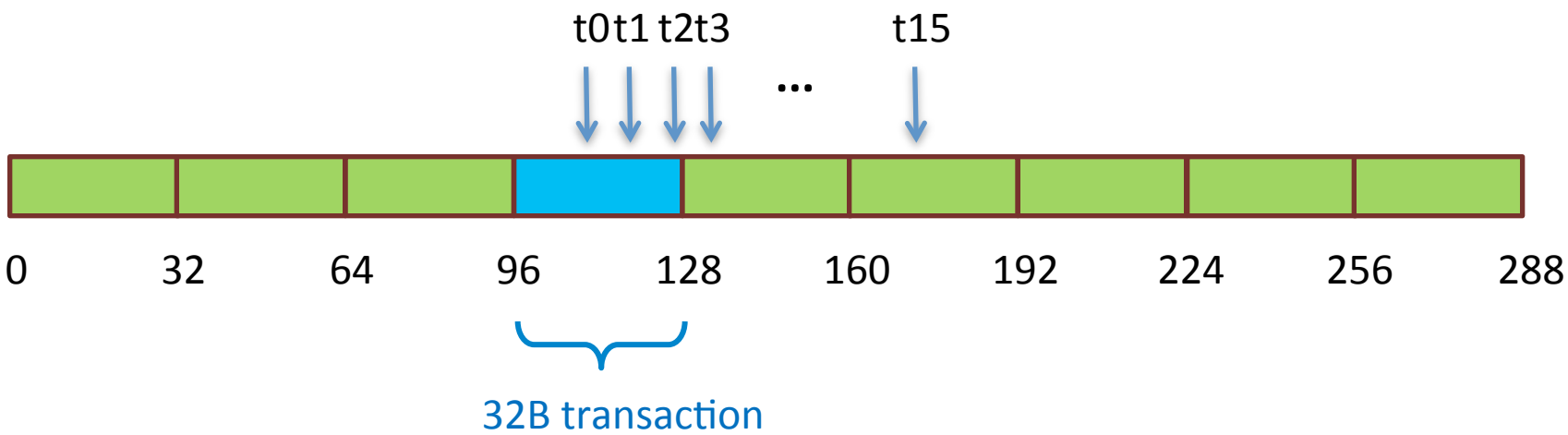
Threads 0-15 access 4-byte words at addresses 116-176

- Thread 0 is lowest active, accesses address 116
- 128-byte segment: 0-127 (reduce to 64B)



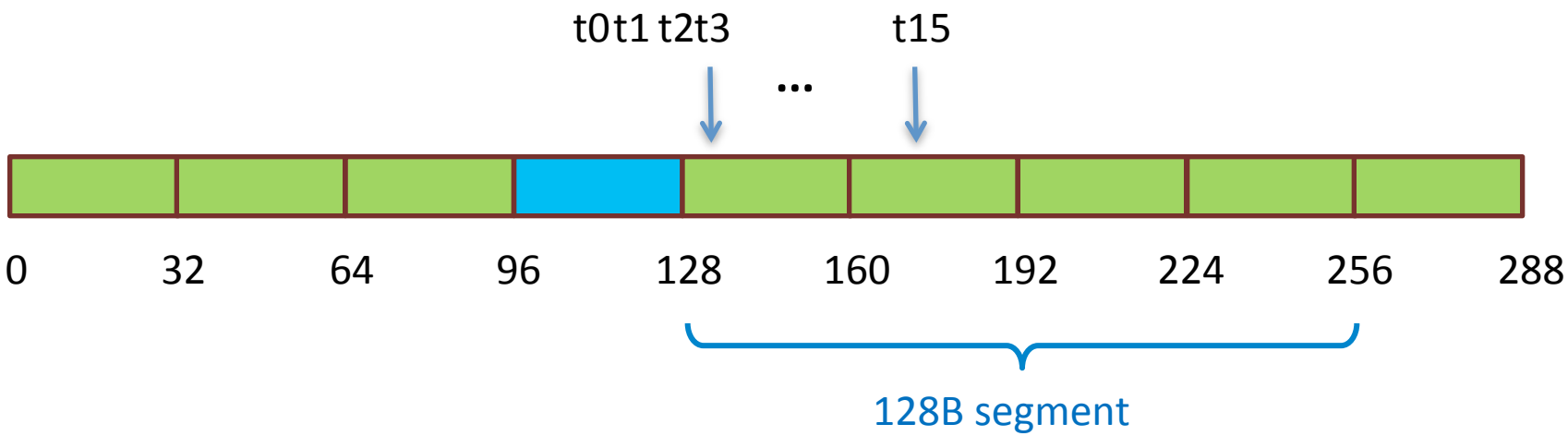
Threads 0-15 access 4-byte words at addresses 116-176

- Thread 0 is lowest active, accesses address 116
- 128-byte segment: 0-127 (reduce to 32B)



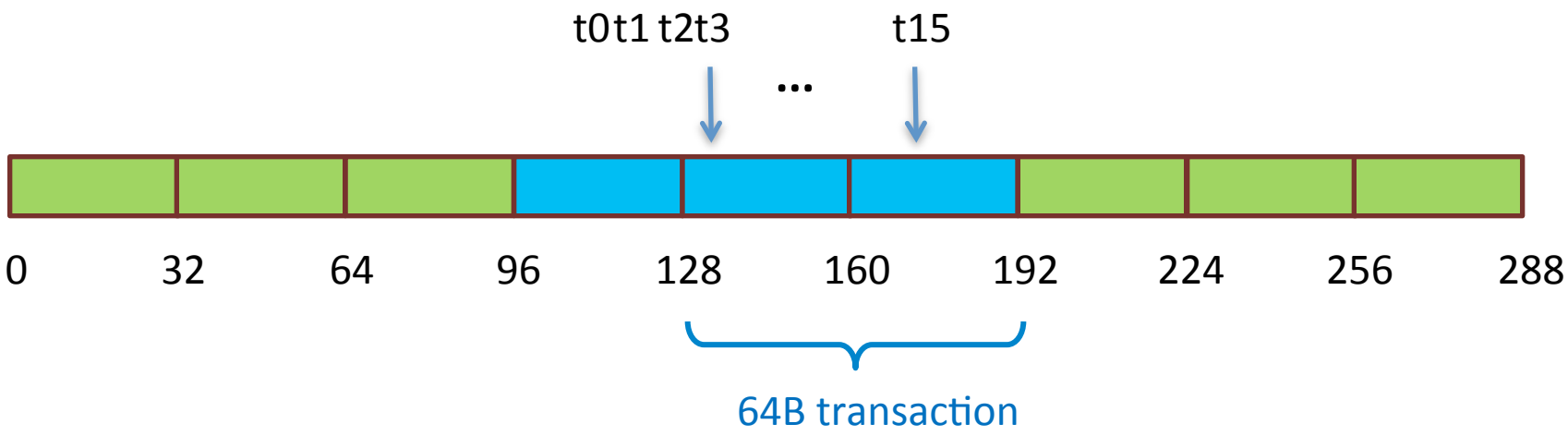
Threads 0-15 access 4-byte words at addresses 116-176

- Thread 3 is lowest active, accesses address 128
- 128-byte segment: 128-255



Threads 0-15 access 4-byte words at addresses 116-176

- Thread 3 is lowest active, accesses address 128
- 128-byte segment: 128-255 (reduce to 64B)



Memory Coalescing

- Hence, the correct way to do it is just to have consecutive threads access consecutive memory addresses.